

UNIVERSITY OF PISA

DEPARTMENT OF COMPUTER SCIENCE

Kinds in Haskell and Their Analogues in F# and JavaScript

Verification log

Course:

Advanced Programming

Student:

Claudio Bernardoni

617214

ACADEMIC YEAR 2025/2026

Contents

1 Verification Methodology **2**

2 Verified Corrections **2**

Verification 1: Typing examples in Haskell 2
Verification 2: Misleading statement regarding kind grammar 2
Verification 3: Unending rule system 3
Verification 4: Common practice claimed as required usage 3
Verification 5: Imprecise claim about multiple declarations with the same name 4
Verification 6: Typing examples in Haskell 4
Verification 7: Clarification about a difference in GHC and GHCi 5
Verification 8: Overstated OCaml restrictions 5
Verification 9: Incorrect parallelism between TypeScript and Haskell 6
Verification 10: Misleading sentence about standard TypeScript 6

References **7**

1 Verification Methodology

Generative AI was heavily used to create the examples and explanations used in the report; however, its outputs cannot be assumed to be correct. Large language models generate text by predicting plausible continuations based on patterns in their training data; as a consequence, an answer may appear convincing while containing incorrect terminology or even factual inaccuracies.

For this reason, the AI-generated material used during the development of this project was independently verified before being incorporated into the final manuscript. Claims about programming languages, type systems, syntax, and language features were compared against authoritative sources, while code examples were checked by executing them.

The purpose of this document is to record the most relevant cases in which an AI-generated statement required correction. For each case, the original incorrect or misleading statement is reported, followed by the corrected version and, if needed, the source used to verify the correction.

2 Verified Corrections

Verification 1: Typing examples in Haskell

Incorrect statement

```
[1, 2, 3]    :: [Int]
Left "oops"  :: Either String Int
```

When providing the typing examples for some values, the AI committed one error and one minor mistake, the latter being the type of the value `[1, 2, 3]`, which is not `[Int]` but `Num a => [a]`. Regarding the other example, i.e. `Left "oops"`, its type is not `Either String Int` because no information is given on the other type of `Either`.

Correction

```
[True, False] :: [Bool]
Left "oops"   :: Either String b
```

The first example was substituted with a list of boolean values `[True, False] :: [Bool]`, which, other than having the right type, avoids having to talk about `Constraints` too soon in the report, when the reader still has to get accustomed to kinds. Regarding the other example, it now provides the right type for the value expressed.

Error location: Section 1.1

Verification 2: Misleading statement regarding kind grammar

Incorrect statement

> *Kinds are generated by a very small grammar:*
>
> $$\kappa \ ::= \ \mathsf{Type} \ \mid \ \kappa_1 \to \kappa_2$$

The sentence is too absolute, making it seem like this simple grammar generates all the possible kinds in Haskell. This is also later contradicted when talking about `Constraints`.

Correction

> *At their core, kinds are generated by this minimal grammar:*

The sentence is not absolute anymore and highlights that this grammar is just the core of kind-like reasoning.

Error location: Section 1.2

Verification 3: Unending rule system

Incorrect statement

> *The kind of a compound type expression is computed by a single rule, the type-level counterpart of the typing rule for function application:*

$$\frac{t_1 :: \kappa_1 \to \kappa_2 \qquad t_2 :: \kappa_1}{t_1\ t_2 :: \kappa_2}\ \text{K-App}$$

The rule system provided consists of just one rule, but any rule system without axioms does not allow for the inference process to stop.

Correction

> *Kinds are assigned by two rules. Let Γ be the kind environment: it records the kind of every type constructor in scope. [...]*

$$\frac{(T :: \kappa) \in \Gamma}{\Gamma \vdash T :: \kappa}\ \text{K-Con} \qquad \frac{\Gamma \vdash t_1 :: \kappa_1 \to \kappa_2 \qquad \Gamma \vdash t_2 :: \kappa_1}{\Gamma \vdash t_1\ t_2 :: \kappa_2}\ \text{K-App}$$

The unstated axiom K-Con is added to the rule system, together with the environment Γ. With these additions, the inference process is allowed to terminate.

Error location: Section 1.2

Verification source

- **Source:** "Types and programming languages" - Benjamin C. Pierce
- **Relevant section:** Chapter 29

Although the documentation contains a richer version of the rule system that surpasses the needs of the report, it confirms the need for the K-Con axiom so that the inference process can finish.

Verification 4: Common practice claimed as required usage

Incorrect statement

> *Note that `Result` lists the success type first, the opposite order from `Either`.*

The AI presents the parameter order of `Either` as if it were an intrinsic property of the type constructor, without distinguishing the type's definition from its conventional use for representing computations that may fail. Although Haskell code conventionally uses `Either e a` to represent an error of type `e` in `Left` or a successful result of type `a` in `Right`, `Either` does not enforce this interpretation. Its two type parameters can represent any pair of types, and their intended meanings depend on how the programmer uses them.

Correction

> *The parameter order of `Result` also differs from that of `Either`: `Result<'T,'TError>` takes the success type first. `Either` itself is neutral, but by convention Haskell code puts*

the successful result in `Right`, [...]

The correction makes the comparison precise without implying that Haskell's `Either` mandates a particular semantic interpretation for its parameters. The distinction is relevant when comparing the two types as representations of computations that can succeed or fail, but it should be presented as a difference in conventional parameter usage rather than as an inherent limitation of `Either`.

Error location: Section 2.1

Verification 5: Imprecise claim about multiple declarations with the same name

Incorrect statement

A Haskell module could not declare two type constructors with the same name and different kinds.

The wording makes it sound that, although it is possible to declare two type constructor with the same name in Haskell, they must have the same kind. Since this is not true, the sentence was changed.

Correction

In Haskell, by contrast, a module cannot declare two type constructors with the same name at all, so a name determines a single constructor, and with it a single kind.

The correction clarifies that in Haskell it is not possible to declare two type constructor with the same name to begin with; therefore, it is not possible even if they have a different kind.

Error location: Section 2.2

Verification 6: Typing examples in Haskell

Incorrect statement

```
// F#
type Validated<'T> = Result<'T, string>
```

The abbreviation looks like a partial application, and in one respect it is more flexible: it fixes the second argument of `Result`, which Haskell's type syntax cannot do directly, since there is no type-level lambda and partial application only fixes leading arguments. An F# abbreviation is in effect a named type-level function, `Validated` = `λT.Result⟨T, string⟩`, and it may fix any of the arguments.

Claiming that Haskell's type syntax cannot fix the second argument of `Either` is technically incorrect. In fact, `type Checked a = Either a String` is legal Haskell, fixing the second argument without constraining the first.

Correction

The abbreviation looks like a partial application, but its Haskell counterpart is a different construct: a type synonym with a parameter, such as `type Checked a = Either a String`.

Now it is specified that partial application on the second argument of `Either` in Haskell is possible, although a different construct is used with respect to F#.

Error location: Section 2.4

Verification 7: Clarification about a difference in GHC and GHCi

Incorrect statement

> *F# has the equivalent of Haskell's type synonyms (which in Haskell, too, must normally be fully applied)*

While it is true that in F# type synonyms must be fully applied, the situation in Haskell is a bit more complex. In GHC, the standard Haskell compiler, it is true that type synonyms must be fully applied; however, it is important to note that the `:kind` command in GHCi is more permissive and also allows for partial application of type synonyms.

Correction

> *GHCi's* `:kind` *command is more permissive than the language: for* `type Either2 a b = Either a b` *it accepts* `:kind Either2 Int` *and reports* `Type -> Type`*, although a program that uses* `Either2 Int` *unsaturated, e.g. in* `Functor (Either2 Int)`*, is rejected because the synonym* `Either2` *should have 2 arguments.*

This particular GHCi behaviour is now clarified in a footnote.

Error location: Section 2.4

Verification source

The correction was verified using the following source:
- **Source:** Haskell documentation, GHCi commands.
- **Relevant section:** `:kind` command.
- **Link:** https://downloads.haskell.org/ghc/7.8.4/docs/html/users_guide/ghci-commands.html
- **Accessed:** 23 October 2026.

Directly from the source:

> `:kind` *even allows you to write a partial application of a type synonym (usually disallowed)*

Verification 8: Overstated OCaml restrictions

Incorrect statement

> *This technique, known as* lightweight higher-kinded polymorphism*, was introduced for OCaml, which has the same restriction [as F#].*

While it is true that OCaml's core has the same restrictions as F# regarding functors, Ocaml as a language can abstract over type constructors natively, through its module system.

Correction

> *This technique, known as* lightweight higher-kinded polymorphism*, was introduced for OCaml, whose core language has the same restriction, although its module system can abstract over type constructors through functors, which F# lacks.*

OCaml's restrictions regarding functors is not overstates anymore.

Error location: Section 2.7

Verification source

The correction was verified using the following source:
- **Source:** OCaml documentation, functors.
- **Link:** https://ocaml.org/docs/functors
- **Accessed:** 23 October 2026.

Verification 9: Incorrect parallelism between TypeScript and Haskell

Incorrect statement

```
type V = Result<string>;                    // TS2314
```

> *The errors on `ys`, `V` and `P` correspond to the first two kind checks of Section 1.5 and have the same practical effect, exactly as in Section 2.3.*

While the sentence is true for the examples of `ys` and `P`, it isn't for `V`. This is because that example shows an instance of higher-kinded partial application, which does not contradict any of the kind checks of Haskell described in Section 1.5. The problem stems from the fact that higher-kinded partial application is not allowed in TypeScript.

Correction

> *The error on `V` is different in nature: the corresponding Haskell synonym `type V = Either String` is legal, so this line shows the absence of partial application rather than a kind error.*

The error instance of `V` is now properly labeled. It does not fail due to similar reason to those of kind checks in Haskell, but because of the absence of higher-kinded reasoning in TypeScript.

Error location: Section 3.2

Verification 10: Misleading sentence about standard TypeScript

Incorrect statement

> *The standard TypeScript answer is the defunctionalisation already met in Section 2.7 (lightweight higher-kinded polymorphism): a type constructor is represented by a name, and application by a table that maps names to types.*

The wording of this sentence makes it seem like the TypeScript standard already possesses lightweight higher-kinded polymorphism. However, that is not true, the actual source from which the AI drew the code it showed is an open-source third party library.

Correction

> *TypeScript has no feature for this; the usual technique is a programming pattern, the defunctionalisation already met in Section 2.7 (lightweight higher-kinded polymorphism): a type constructor is represented by a name, and application by a table that maps names to types.*

Now the actual TypeScript situation regarding higher-kinded polymorphism is stated correctly and unambiguously.

Error location: Section 3.3

References

[1] G. Canti. *fp-ts*: typed functional programming in TypeScript. Source code and documentation (module `HKT`). https://github.com/gcanti/fp-ts, accessed October 2026.

[2] M. P. Jones. A system of constructor classes: overloading and implicit higher-order polymorphism. *Journal of Functional Programming*, 5(1):1–35, 1995. DOI 10.1017/S0956796800001210.

[3] S. Marlow (editor). *Haskell 2010 Language Report*, 2010. https://www.haskell.org/onlinereport/haskell2010/.

[4] Microsoft. F# Language Reference. Microsoft Learn documentation. https://learn.microsoft.com/en-us/dotnet/fsharp/language-reference/, accessed October 2026.

[5] Microsoft. The TypeScript Handbook. https://www.typescriptlang.org/docs/handbook/, accessed October 2026.

[6] OCaml.org. Functors. OCaml documentation. https://ocaml.org/docs/functors, accessed October 2026.

[7] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.

[8] D. Syme. The early history of F#. *Proceedings of the ACM on Programming Languages*, 4(HOPL), Article 75, 2020. DOI 10.1145/3386325.

[9] The GHC Team. *Glasgow Haskell Compiler User's Guide*, version 9.14.1. https://downloads.haskell.org/ghc/latest/docs/users_guide/, accessed October 2026.

[10] J. Yallop and L. White. Lightweight higher-kinded polymorphism. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475 of *Lecture Notes in Computer Science*. Springer, 2014. DOI 10.1007/978-3-319-07151-0_8.